

Practical no:1

Q1. Create an HTML form for Login and write a JavaScript to validate email ID and Password using Regular Expression.

Steps to Create:

1. Create folder: `javascript`
2. Create file: `login.html`

Code (login.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <script>
    let validation=()=>{
      let isvalid=true;
      var emailV=document.getElementById("email").value.trim();
      var password=document.getElementById("password").value.trim();

      document.getElementById("emailError").innerText="";
      document.getElementById("passwordError").innerText="";

      let regEmail=/^[a-z0-9._-]+@[a-z]+\.[a-z]{2,3}$/;
      let regPassword=/^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9]).{6,}$/

      if(!regEmail.test(emailV)){
        document.getElementById("emailError").innerText="Enter the valid email";
        isvalid=false;
      }
      if(!regPassword.test(password)){
        document.getElementById("passwordError").innerText="Enter the valid password";
        isvalid=false;
      }
      if(isvalid){
        alert("Form succesfully submit");
      }
      return isvalid;
    }
  </script>
  <title>Login form</title>
</head>
<body>
  <form align="center" style="border:2px solid black; padding: 20px;" onsubmit="return validation()">
    <h2>Login</h2>
    <label>Name</label>
    <input type="text" id="name" placeholder="Enter the name"/><br>
    <label>Email</label>
    <input type="text" id="email" placeholder="Enter the email"/><br>
    <span id="emailError"></span><br>
    <label>Password</label>
    <input type="password" id="password" placeholder="Enter the password"/><br>
    <span id="passwordError"></span><br>
    <button type="submit">Submit</button>
  </form>
```

</body>
</html>

Steps to Execute:

1. Open 'login.html' in a browser.
2. Enter valid/invalid details to test validation.

Output: -

Login Name Jagdish Email jaggu@gmail.com Password zhujiA123  Submit	127.0.0.1:5500 says Form succesfully submit OK Email jaggu@gmail.com Password Submit
--	--

Login

Name

Email

Password 

Signature of instructor

Date

Practical no:2

Q2. Create an HTML form that contain the Student Registration details and write JavaScript to validate Student first and last name as it should not contain other than alphabets and age should be between 18 to 50.

Steps to Create:

1. Create folder: `javascript`
2. Create file: `student.html`

Code (student.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Registration Form</title>
  <script>
    const validateForm = () => {
      let isValid = true;
      const firstName = document.getElementById('firstName').value.trim();
      const lastName = document.getElementById('lastName').value.trim();
      const age = document.getElementById('age').value.trim();
      const firstNameError = document.getElementById('firstNameError');
      const lastNameError = document.getElementById('lastNameError');
      const ageError = document.getElementById('ageError');
      firstNameError.textContent = "";
      lastNameError.textContent = "";
      ageError.textContent = "";
      let reF=/^[A-Za-z]+$/.;
      if (!reF.test(firstName)) {
        firstNameError.textContent = 'First name should only contain alphabets.';
        isValid = false;
      }
      if (!reF.test(lastName)) {
        lastNameError.textContent = 'Last name should only contain alphabets.';
        isValid = false;
      }
      if (!age || age < 18 || age > 50) {
        ageError.textContent = 'Age must be between 18 and 50.';
        isValid = false;
      }
      if (isValid) {
        alert('Registration successful!');
        document.getElementById('registrationForm').reset();
      }
    };
  </script>
</head>
<body>
  <h2>Student Registration</h2>
  <form id="registrationForm">
    <label for="firstName">First Name:</label>
    <input type="text" id="firstName" placeholder="Enter your first name">
    <span id="firstNameError" style="color:red"></span><br><br>
    <label for="lastName">Last Name:</label>
    <input type="text" id="lastName" placeholder="Enter your last name">
    <span id="lastNameError" style="color:red"></span><br><br>
```

```

<label for="age">Age:</label>
<input type="number" id="age" placeholder="Enter your age">
<span id="ageError" style="color:red"></span><br><br>
<button type="button" onclick="validateForm()">Register</button>
</form>
</body>
</html>

```

Steps to Execute:

1. Open `student.html` in browser.
2. Enter student details to test name and age validation.

Output:-

Student Registration

127.0.0.1:5500 says
Registration successful!

First Name:

Last Name:

Age:

Student Registration

First Name: First name should only contain alphabets.

Last Name: Last name should only contain alphabets.

Age: Age must be between 18 and 50.

Signature of instructor

Date

Practical no:3

Q3. Create a Node.js file that will convert the output "Full Stack!" into reverse string.

Steps to Create:

1. Create folder: `nodejs`
2. Create file: `slip2.js`

Code (slip2.js):

```
const http = require('http');
const PORT = 3011;
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  const input = "Full Stack!";
  const reversed = input.split("").reverse().join("");
  res.end(`Reversed string: ${reversed}`);
});
server.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

Steps to Execute:

1. Open terminal in `nodejs` folder.
2. Run: `node slip2.js`
3. Access `http://localhost:3011` in browser.

Output:-

```
Input: Full Stack!
Reversed string: !kcatS lluF
```

Signature of instructor

Date

Practical no:4

Q4. Using node js create a web page to read two file names from the user and append contents of the first file into the second file.

Steps to Create:

1. Create folder: `nodejs`
2. Create file: `slip2b.js`
3. Create sample files `a.txt` and `b.txt` in the same directory.

Code (slip2b.js):

```
const http = require('http');
const fs = require('fs');
const { parse } = require('querystring');
const server = http.createServer((req, res) => {
  if (req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end(`
      <h2>File Appender</h2>
      <form method="POST">
        <input type="text" name="source" placeholder="Source File (e.g., a.txt)" required><br><br>
        <input type="text" name="dest" placeholder="Destination File (e.g., b.txt)" required><br><br>
        <button type="submit">Append Contents</button>
      </form>
    `);
  } else if (req.method === 'POST') {
    let body = "";
    req.on('data', chunk => { body += chunk.toString(); });
    req.on('end', () => {
      const { source, dest } = parse(body);
      fs.readFile(source, 'utf8', (err, data) => {
        if (err) {
          res.writeHead(404, { 'Content-Type': 'text/html' });
          return res.end(`<h3>Error: Source file "${source}" not found!</h3><a href="/">Try Again</a>`);
        }
        fs.appendFile(dest, '\n' + data, (appendErr) => {
          if (appendErr) {
            res.writeHead(500, { 'Content-Type': 'text/html' });
            return res.end(`<h3>Error: Could not write to "${dest}"</h3>`);
          }
          res.writeHead(200, { 'Content-Type': 'text/html' });
          res.end(`<h3>Success!</h3><p>Contents added.</p><a href="/">Back</a>`);
        });
      });
    });
  }
});
server.listen(3000, () => console.log('Server on 3000'));
```

Steps to Execute:

1. Run: `node slip2b.js`
2. Open `http://localhost:3000`, enter `a.txt` and `b.txt`, then click Append.

Output:

Before -

```
lab > nodejs > a.txt
1 This is source file content.

lab > nodejs > b.txt
1 This is destination file content.
```

After -

File Appender

```
lab > nodejs > a.txt
1 This is source file content.

lab > nodejs > b.txt
1 This is destination file content.
2 This is source file content.
```

Signature of instructor

Date

Practical no:5

Q5. Create an HTML form using AngularJS that contain the Student Registration details and validate Student first and last name as it should not contain other than alphabets and age should be between 18 to 50 and display greeting message depending on current time using ng-show (e.g. Good Morning, Good Afternoon, etc.)(Use AJAX).

Steps to Create:

1. Create file: `angular\_student.html`
2. Include AngularJS script via CDN.

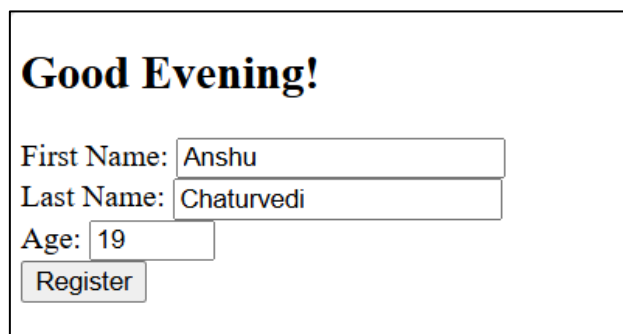
Code (angular_student.html):

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<body ng-app="myApp" ng-controller="myCtrl">
<h2 ng-show="isMorning">Good Morning!</h2>
<h2 ng-show="isAfternoon">Good Afternoon!</h2>
<h2 ng-show="isEvening">Good Evening!</h2>
<form name="studentForm" novalidate>
  First Name: <input type="text" name="f" ng-model="f" ng-pattern="/^[a-zA-Z]*$/" required><br>
  Last Name: <input type="text" name="l" ng-model="l" ng-pattern="/^[a-zA-Z]*$/" required><br>
  Age: <input type="number" ng-model="age" min="18" max="50" required><br>
  <button ng-disabled="studentForm.$invalid">Register</button>
</form>
<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  var h = new Date().getHours();
  $scope.isMorning = h < 12;
  $scope.isAfternoon = h >= 12 && h < 17;
  $scope.isEvening = h >= 17;
});
</script>
</body>
</html>
```

Steps to Execute:

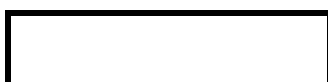
Open `angular\_student.html` in browser.

Output:



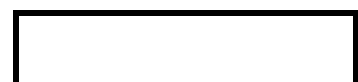
The screenshot shows a web browser displaying the output of the AngularJS application. At the top, a large heading reads "Good Evening!". Below this, there is a registration form with four input fields: "First Name:" containing "Anshu", "Last Name:" containing "Chaturvedi", and "Age:" containing "19". A "Register" button is positioned below the age field. The form is styled with a light gray border and a white background.

Signature of instructor



A rectangular box with a black border, intended for the instructor's signature.

Date



A rectangular box with a black border, intended for the date.

Practical no:6

Q6. Create a Node.js file that writes an HTML form, with an upload field.

Steps to Create:

1. Create folder: `nodejs`
2. Create file: `upload.js`
3. Install formidable: `npm install formidable`

Code (upload.js):

```
var http = require('http');
var formidable = require('formidable');
var fs = require('fs');
http.createServer(function (req, res) {
  if (req.url === '/fileupload') {
    var form = new formidable.IncomingForm();
    form.parse(req, function (err, fields, files) {
      res.write('File uploaded!');
      res.end();
    });
  } else {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write('<form action="fileupload" method="post" enctype="multipart/form-data">');
    res.write('<input type="file" name="filetoupload"><br><input type="submit"></form>');
    res.end();
  }
}).listen(8080, () => console.log("Server on 8080"));
```

Steps to Execute:

1. Run: `node upload.js`
2. Open `http://localhost:8080`, select a file and upload.

Output:

Choose File Slip No 5(1q).txt Submit	← ↻ ⓘ localhost:8080/fileupload File uploaded!
--	--

Signature of instructor

Date

Practical no:7

Q7. Using angular js create a SPA to carry out validation for a username entered in a textbox. If the textbox is blank, alert “Enter username”. If the number of characters is less than three, alert “Username is too short”. If the value entered is appropriate the print “Valid username” and password should be a minimum of 8 characters.

Steps to Create:

1. Install Angular CLI: `npm install -g @angular/cli`
2. Create project: `ng new slip5 --defaults`
3. Folder structure: `slip5/src/app/`

Code (src/app/app.component.ts):

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  username: string = "";
  password: string = "";
  message: string = "";
  validateForm() {
    if (this.username.trim() === "") {
      alert("Enter username");
      return;
    }
    if (this.username.length < 3) {
      alert("Username is too short");
      return;
    }
    if (this.password.length < 8) {
      alert("Password must be minimum 8 characters");
      return;
    }
    this.message = "Valid username";
  }
}
```

Code (src/app/app.component.html):

```
<h2>User Validation SPA</h2>
<form (ngSubmit)="validateForm()">
<label>Username:</label><br>
<input type="text" [(ngModel)]="username" name="username" required><br><br>
<label>Password:</label><br>
<input type="password" [(ngModel)]="password" name="password" required><br><br>
<button type="submit">Validate</button>
</form>
<p>{{message}}</p>
```

Steps to Execute:

1. Navigate to `slip5` folder.
2. Run: `ng serve`
3. Open `http://localhost:4200`

Output :-

User Validation Username: Password: Validate	localhost:4200 says Enter username OK
User Validation Username: fr Password: Validate	localhost:4200 says Username is too short OK
User Validation Username: frgrrehfdh Password: *** Validate	localhost:4200 says Password must be minimum 8 characters OK
User Validation SPA Username: frgrrehfdh Password: ***** Validate Valid username	

Signature of instructor

Date

Practical no:8

Q8. Write angular JS by using ng-click directive to display an alert message after clicking the element.

Steps to Create:

1. Create project: `ng new message --defaults`
2. Navigate to: `nodejs/message/src/app/`

Code (src/app/app.component.html):

```
<button (click)="showAlert()">Click Me</button>
```

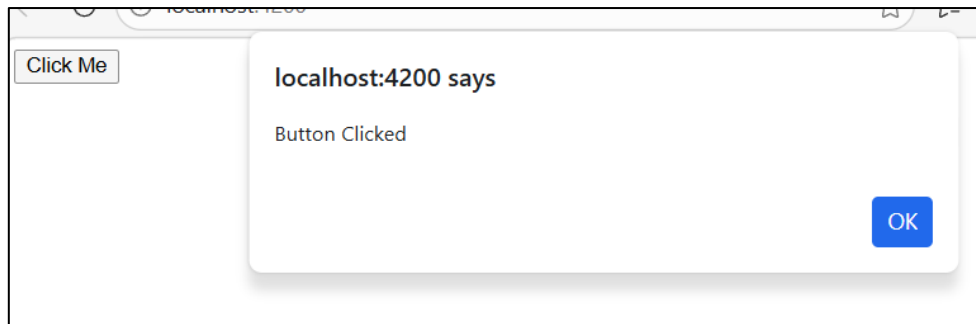
Code (src/app/app.component.ts):

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  showAlert() {
    alert("Button Clicked")
  }
}
```

Steps to Execute:

1. Run: `ng serve`
2. Click button to see alert.

Output :-



Signature of instructor

Date

Practical no:9

Q9. Create angular JS Application that show the current Date and Time of the System (Use Interval Service)

Steps to Create

1. Create file: `interval.html`
2. Include AngularJS CDN.

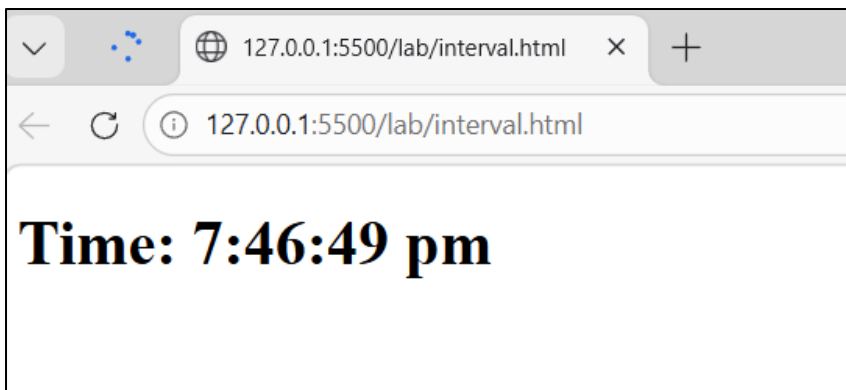
Code (interval.html):

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<body ng-app="myApp" ng-controller="myCtrl">
<h1>Time: {{ theTime }}</h1>
<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope, $interval) {
    $interval(function () {
        $scope.theTime = new Date().toLocaleTimeString();
    }, 1000);
});
</script>
</body>
</html>
```

Steps to Execute:

Open `interval.html` in browser.

Output:



Signature of instructor

Date

Practical no:10

Q10. Create a node js file named main.js for event-driven application. There should be a main loop that listens for events, and then triggers a callback function when one of those events is detected. Create a Simple Web Server using node js.

Steps to Create:

1. Create file: `nodejs/slip7.js` (Event Driven)
2. Create file: `nodejs/slip14.js` (Server)

Code (nodejs/slip7.js):

```
var events = require('events');
var EventEmitter = new events.EventEmitter();
eventEmitter.on('data_received', (name) => {
  console.log(name, "Understood event -Driven");
});
eventEmitter.emit('data_received', "FYMSC(CS)");
console.log("program Ended");
```

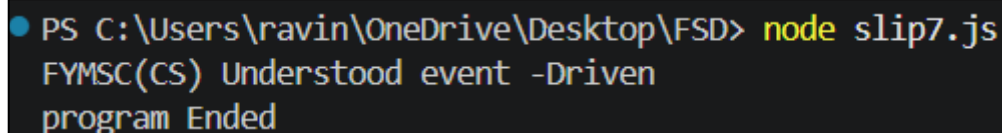
Code (nodejs/slip14.js):

```
var http=require('http');
http.createServer(function(req,res){
  res.writeHead(200,{'Content-Type':'text/html'});
  res.write('Hello world!');
  res.end();
}).listen(3000, () => console.log('Active port 3000'));
```

Steps to Execute:

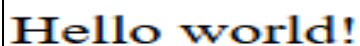
1. Run: `node slip7.js`
2. Run: `node slip14.js` and visit `http://localhost:3000`.

Output: Slip7



```
PS C:\Users\ravin\OneDrive\Desktop\FSD> node slip7.js
FYMSC(CS) Understood event -Driven
program Ended
```

Output: Slip14



```
Hello world!
```

Signature of instructor

Date

Practical no:11

Q11. Using angular js display the 10 student details in Table format (using ng-repeat directive use Array to store data)

Steps to Create:

1. Create file: `table.html`
2. Use AngularJS CDN.

Code (table.html):

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>

<body ng-app="myApp" ng-controller="myCtrl">

<table border="1">
  <tr ng-repeat="s in students">
    <td>{{ s.name }}</td>
    <td>{{ s.course }}</td>
  </tr>
</table>

<script>
var app = angular.module('myApp', []);

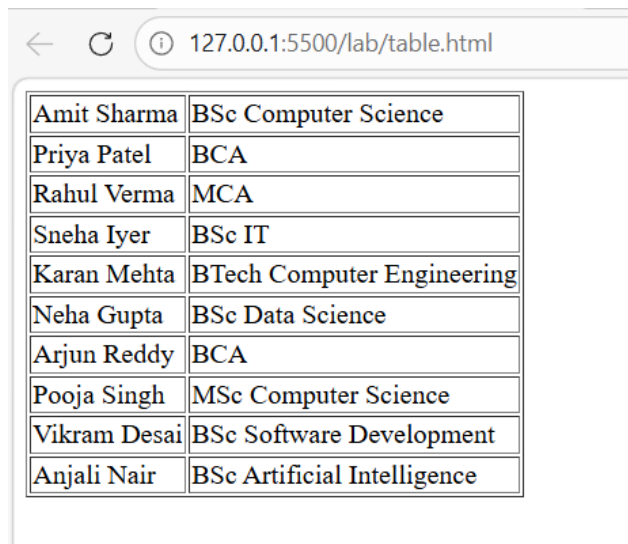
app.controller('myCtrl', function($scope) {
  $scope.students = [
    {name:'Amit Sharma', course:'BSc Computer Science'},
    {name:'Priya Patel', course:'BCA'},
    {name:'Rahul Verma', course:'MCA'},
    {name:'Sneha Iyer', course:'BSc IT'},
    {name:'Karan Mehta', course:'BTech Computer Engineering'},
    {name:'Neha Gupta', course:'BSc Data Science'},
    {name:'Arjun Reddy', course:'BCA'},
    {name:'Pooja Singh', course:'MSc Computer Science'},
    {name:'Vikram Desai', course:'BSc Software Development'},
    {name:'Anjali Nair', course:'BSc Artificial Intelligence'}
  ];
});
</script>

</body>
</html>
```

Steps to Execute:

Open `table.html` in browser.

Output:



Amit Sharma	BSc Computer Science
Priya Patel	BCA
Rahul Verma	MCA
Sneha Iyer	BSc IT
Karan Mehta	BTech Computer Engineering
Neha Gupta	BSc Data Science
Arjun Reddy	BCA
Pooja Singh	MSc Computer Science
Vikram Desai	BSc Software Development
Anjali Nair	BSc Artificial Intelligence

Signature of instructor

Date

Practical no:12

Q12. Create a Node.js file that writes an HTML form, with a concatenate two string.

Steps to Create:

1. Create file: `nodejs/slip9.js`

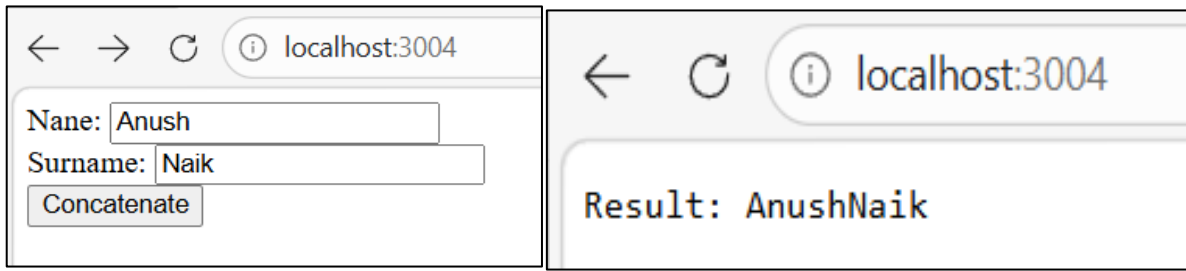
Code (nodejs/slip9.js):

```
const http = require('http');
const { parse } = require('querystring');
const server = http.createServer((req, res) => {
  if (req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<form method="POST">Name: <input name="str1"> <br>Surname: <input name="str2"> <br><button>Concatenate</button></form>');
  } else {
    let body = "";
    req.on('data', c => body += c);
    req.on('end', () => {
      const formData = parse(body);
      res.end("Result: " + (formData.str1 + formData.str2));
    });
  }
}).listen(3004, () => console.log('Server on 3004'));
```

Steps to Execute:

1. Run: `node slip9.js`
2. Open `http://localhost:3004`

Output:



Signature of instructor

Date

Practical no:13

Q13. Write node js script to build Your Own Node.js Module. Use require ('http') module is a built in Node module that invokes the functionality of the HTTP library to create a local server. Also use the export statement to make functions in your module available externally. Create a new text file to contain the functions in your module called, "modules.js" and add this function.

Steps to Create:

1. Create file: `nodejs/modules.js`
2. Create file: `nodejs/slip10.js`

Code (nodejs/modules.js):

```
function datetime() {  
  let dt = new Date();  
  let date = ("0"+dt.getDate()).slice(-2);  
  let month = ("0"+ (dt.getMonth()+1)).slice(-2);  
  let year = dt.getFullYear();  
  let hours = dt.getHours();  
  let minutes = dt.getMinutes();  
  let seconds = dt.getSeconds();  
  return year + "-" + month + "-" + date + " " + hours + ":" + minutes + ":" + seconds;  
}  
module.exports = {datetime}
```

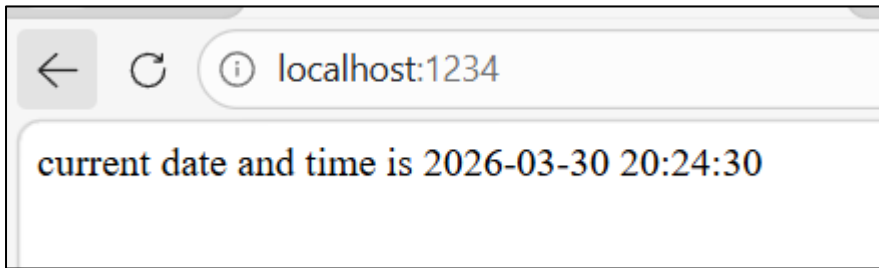
Code (nodejs/slip10.js):

```
var http = require('http');  
var dt = require('./modules');  
var server = http.createServer(function(req,res){  
  res.writeHead(200, {'content-type': 'text/html'});  
  res.write('current date and time is ' + dt.datetime());  
  res.end();  
});  
server.listen(1234, () => console.log("Server on 1234"));
```

Steps to Execute:

1. Run: `node slip10.js`
2. Visit `http://localhost:1234`.

Output:



Signature of instructor

Date

Practical no:14

Q14. Create an angular JS Application that shows the location of the current web page.

Steps to Create:

1. Create file: 'location.html'
2. Use AngularJS CDN.

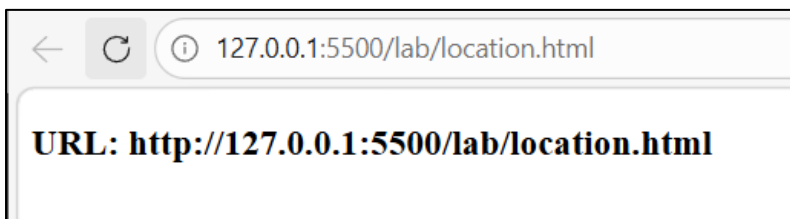
Code (location.html):

```
<!DOCTYPE html>
<html>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<body ng-app="myApp" ng-controller="myCtrl">
<h3>URL: {{myUrl}}</h3>
<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope, $location) {
    $scope.myUrl = $location.absUrl();
});
</script>
</body>
</html>
```

Steps to Execute:

Open 'location.html' in browser.

Output:



Signature of instructor

Date

Practical no:15

Q15. Using angular js create a SPA to accept the details such as name, mobile number, pin code and email address and make validation. Name should contain character only, address should contain SPPU M.Sc. Computer Science Syllabus 2023-24, mobile number should contain only 10 digit, Pin code should contain only 6 digit, email id should contain only one @, . Symbol.

Steps to Create:

1. Create project: `ng new slip19 --defaults`
2. Navigate to: `slip19/src/app/`

Code (src/app/app.component.ts):

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  name = ""; address = ""; mobile = ""; pincode = ""; email = ""; message="";
  validateForm() {
    if(!/^[A-Za-z ]+$/ .test(this.name)) { alert("Name Error"); return; }
    if(!/^[0-9]{10}$/ .test(this.mobile)) { alert("Mobile Error"); return; }
    if(!/^[0-9]{6}$/ .test(this.pincode)) { alert("Pin Error"); return; }
    if(!/[@\./] .test(this.email)) { alert("Email Error"); return; }
    this.message="All Details Validated Successfully";
  }
}
```

Code (src/app/app.component.html):

```
<h2>Student Details Validation</h2>
<form (ngSubmit)="validateForm()">
Name: <input type="text" [(ngModel)]="name" name="n"><br><br>
Address: <textarea [(ngModel)]="address" name="a" required></textarea><br><br>
Mobile: <input type="text" [(ngModel)]="mobile" name="m"><br><br>
Pin: <input type="text" [(ngModel)]="pincode" name="p"><br><br>
Email: <input type="text" [(ngModel)]="email" name="e"><br><br>
<button type="submit">Submit</button>
</form>
<h3>{{message}}</h3>
```

Steps to Execute:

1. Run: `ng serve`
2. Open `http://localhost:4200` to test validations.

Output:

Student Details Validation

Name:

Address:

Mobile:

Pin:

Email:

All Details Validated Successfully

Student Details

Name:

Address:

Mobile:

Pin:

Email:

localhost:4200 says
Name Error

Signature of instructor

Date